(Structures in c++) → user defined data type.

↳ allows to group related data of different data types under a single name.

Struct: A collection of a fixed number of components in which the components are accessed by name

Syntax:

```
Struct  Structname
{
    datatype  identifier 1;
    datatype  identifier 2;
    datatype  identifier 3;
};
```

} called members

↳ this only has defined the data type

Variable declaration:

Structname Variable name; → a single variable that has exactly contains all the members

↳ this declare variables of d.t Structname

→ Memory allocated large enough to store all members of struct.

example

```
Struct Smartphone {
    String name ;
    Int Storagespace ;
    String color ;
    float price ;
};
```

→ all of these variables describe a Smartphone

↳ we dont assign data in struct.

Accessing Struct members.

Structvariable name. member name

note:: Structure is used to store data of diff type and group them in single type

e.g

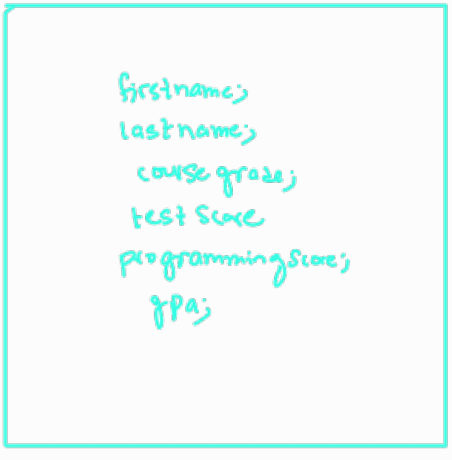
```
Struct  StudentType
{
    String  firstname;
    String  lastname;
    char    courseGrade;
    int     testScore
    int     programmingScore;
    double  gpa;
};
```

```
StudentType newStudent;
StudentType S2
```

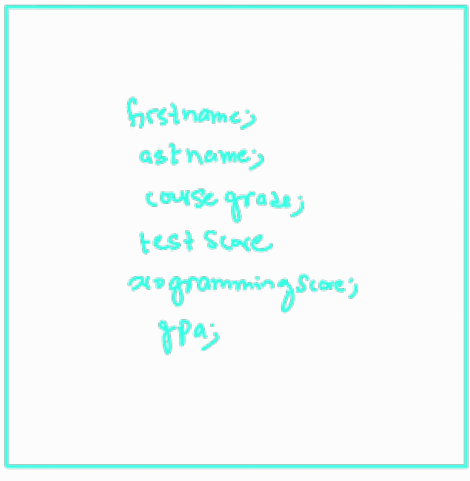
Acessing

```
StudentType.firstname = "John";
cin >> newStudent.firstname;
```

```
cin >> newStudent.testScore >> newStudent.programming score;
```



newStudent



S2

How to pass a Structure to function

```
Void PrintStudentData (StudentType S2)
{
    cout << S2.firstname;
    cout << S2.Secondname
    cout << S2.gpa;
}
```

→ passing Structure Variable

Assigning one structure data to another.

```cpp
#include<iostream>
using namespace std;

struct student {
    int age;
    string name;
    float height;
    char grade;

};

int main()
{
    student s1;
    student s2;

    s1.age = 19;
    s1.grade = 'C';
    s1.height = 6.2;
    s1.name = "Furqan";

    s2 = s1;

    cout << s2.grade;


    return 0;
}
```

here all members of S1 are directly assigned to members of S2

But not the struct variables must have same data type

As here both S1 and S2 are of type student.

Comparision:: we always compare two structures member wise.

```cpp
#include<iostream>
using namespace std;

struct student {
    int age;
    string name;
    float height;
    char grade;

};

int main()
{
    student s1;
    student s2;

    s1.age = 19;
    s1.grade = 'C';
    s1.height = 6.2;
    s1.name = "Furqan";

    s2 = s1;

    if (s1.grade == s2.grade)
        cout << "Yes its the same person";
    else
        cout << "No goddammit";


    return 0;
}
```

This is illegal S2 == S1.

Functions:

Struct variables can be passed by ref or by value.

A function can return a value of type struct.

Passing a struct by ref means it will actually update the structure itself in main. Passing it by value will have no changes made in main.

Note: Always define Struct first. Even before our function prototypes, Int main

---

general Syntax of passing a structure to function

function datatype functionname (Structuretype varname);

↳ Passes all the structure members.

Returning structure from a function.

Structure name (Structuretype varname)

return varname;

```
if (student == newStudent)     //illegal
.
.
.
```

Input/Output

No aggregate input/output operations are allowed on a **struct** variable. Data in a **struct** variable must be read one member at a time. Similarly, the contents of a **struct** variable must be written one member at a time.

We have seen how to read data into a **struct** variable. Let us now see how to output a **struct** variable. The statement:

```
cout << newStudent.firstName << " " << newStudent.lastName
     << " " << newStudent.courseGrade
     << " " << newStudent.testScore
     << " " << newStudent.programmingScore
     << " " << newStudent.GPA << endl;
```

outputs the contents of the **struct** variable newStudent.

struct Variables and Functions

Recall that arrays are passed by reference only, and a function cannot return a value of type array. However:

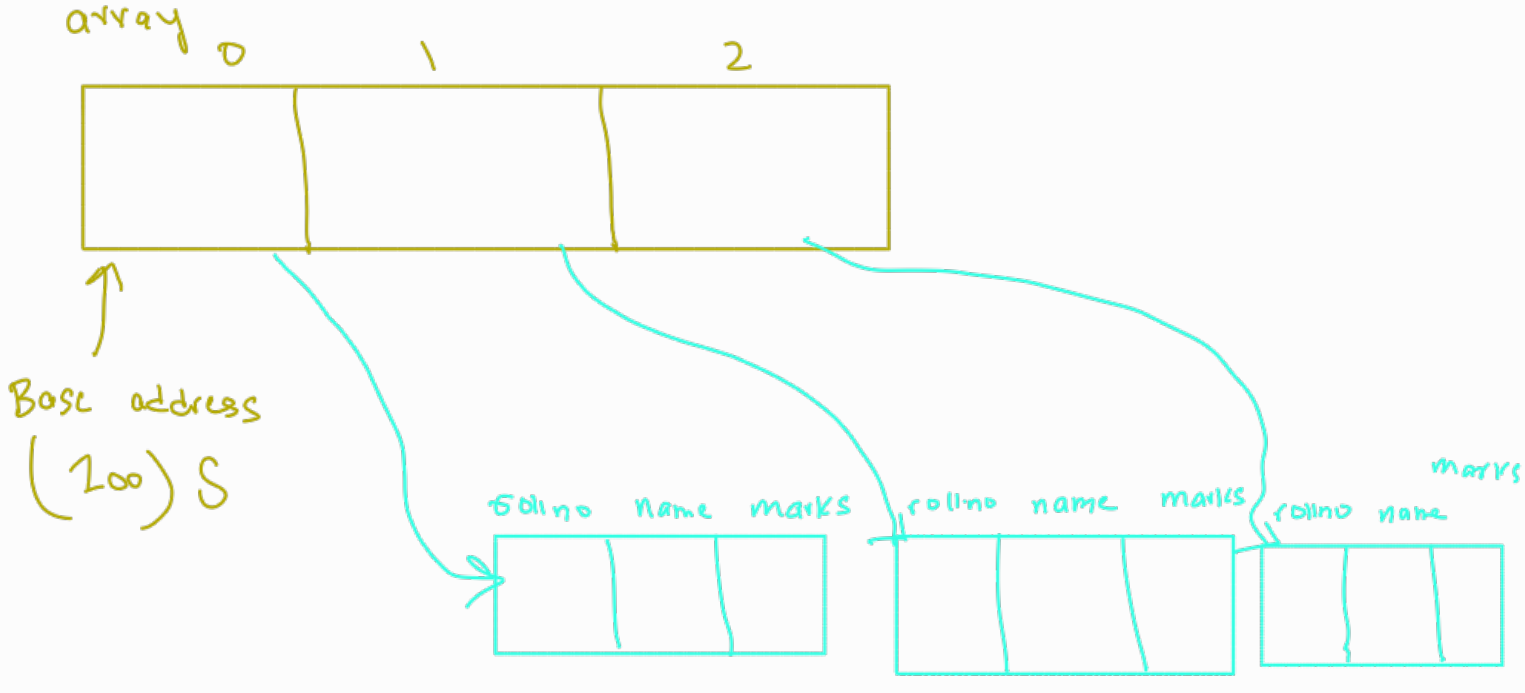
Arrays in Structs

```
Struct Student
{
    Int rollno;
    char name [20];
    float marks;
};
```

```
Student array [3];
```

↓

creates 60 Structures to Store Student data.



Acessing Student 1.data

S[0].rollno.

S[0].marks

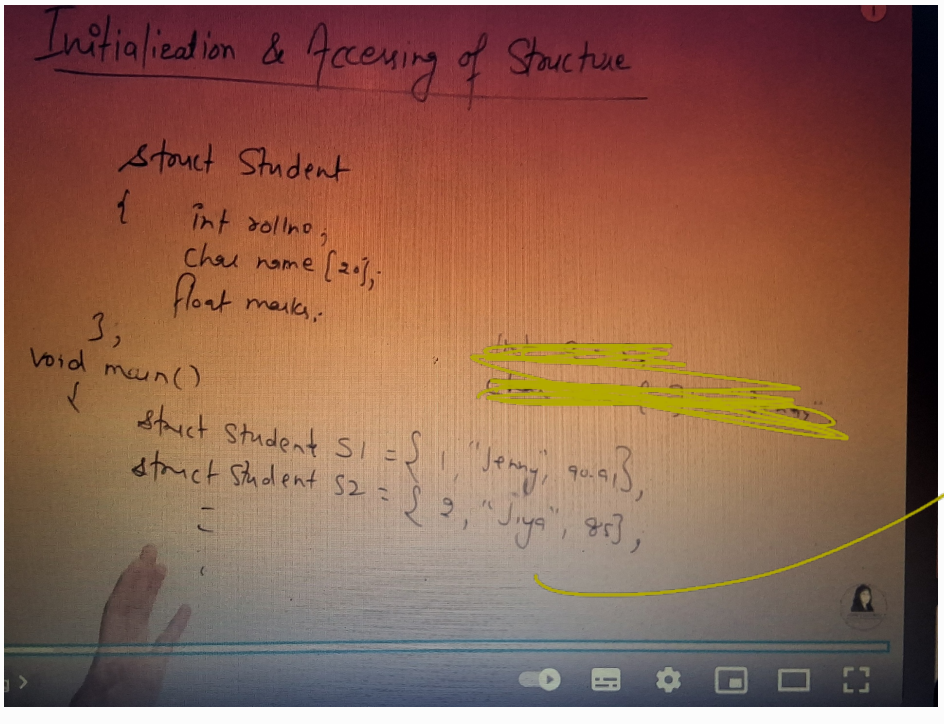
S[0].name

Pass by ref

Pass by val

Pass by const ref

Another way to initialise



order should match

```cpp
#include <iostream> // for cout, cin
using namespace std;

struct Circle
{
    double radius;
    double area;
};

Circle getCircle()
{
    Circle c;
    const double PI = 3.14159;

    cout << "Enter a radius: ";
    cin >> c.radius;

    c.area = c.radius * c.radius * PI;

    return c;
}

int main()
{
```

returning a Structure.

```cpp
struct Circle
{
    double radius;
    double area;
};

double findArea(double r)
{
    const double PI = 3.14159;
    double area = PI * r * r;
    return area;
}

int main()
{
    Circle c;
    c.radius = 1.2;
    c.area = findArea(c.radius);

    cout << "Radius=" << c.radius << "\n";
    cout << "Area=" << c.area << "\n";

    // pause the program
    cin.ignore();
```

Passing members as parameters.